Binary Search Trees

Binary *Search* Trees

As we saw in the previous section: linked lists can provide fast insertion $O(1)$, but searching a linked list is slow $O(n)$; searching an ordered array is fast $O(\log n)$, but inserting in order is slow $O(n)$.

In this section we will look at binary search trees (BSTs). BSTs give us time for insertion, deletion and search of $O(height)$. And the average height of a randomly constructed BST is $O(\log n)$ (actually, not quite $O(\log n)$, but near enough).

So it looks like BSTs are worth looking at.

Content

- Definition
- Search
- Insertion
- Tree Deletion

Definition of a Binary Search Tree (BST)

A BST is a binary tree with the additional property that:

> *At any node in a BST, the values of elements in its left subtree are less than its value, and the values of elements in its right subtree are greater*

Note: for simplicity we will ignore duplicates, but to accommodate duplicates you need to change to 'less than or equal' or 'greater than or equal'.

Searching a BST

Given the BST Order property, finding a value in a BST is fairly straightforward.

> **BST Find**
>
> *Starting at the root:*
>
> - *If the node is empty, return 'not found'*
> - *If the value of the node is greater than the search value, go left*
> - *If the value of the node is less than the search value, go right*

Or, in more formal pseudocode:

> **BST Find**
>
> *def find n in tree:*
> *if tree is empty or tree contains n:*
> *return tree*
> *if n is less than the value in tree:*
> *return find n in left-sub-tree*
> *if n is greater than the value in tree:*
> *return find n in right-sub-tree*

To implement this in C, we will use the following BST definitions:

```
typedef struct bstNode {
    int data_item;
    struct bstNode *left;
    struct bstNode *right;
} *BSTNodePtr;

typedef struct bst {
    BSTNodePtr root;
} BST;
```

Note that some implementations do not use a wrapper class (BST), just the node definition. Using the wrapper allows us to be consistent with the calling conventions of our other data structures, and also simplifies the call (see the recursive call in the insert function later).

With these definitions find can be implemented using the following functions, where find_bst is a wrapper class that should be publicly visible, and find_bst_node is a private helper class that implements the recursive algorithm.

```
BSTNodePtr find_bst_node(BSTNodePtr self, int n){
  if (self == NULL || (n == self->data_item) {
    return self;
  } else if (n < self->data_item) {
    return find_bst_node(self->left, n);
  } else {
    return find_bst_node(self->right, n);
  }
}

BSTNodePtr find_bst(BST *self, int n) {
  ...
```

Inserting into a BST

The process of inserting is similar to searching except that where the search would stop on an empty subtree, the insertion operation adds the element.

> **BST Insert**
>
> *def insert n into tree:*
> *if tree is empty:*
> *add a new node containing n here*
> *else:*
> *if n is less than the value in tree*
> *insert n into left-sub-tree*
> *else:*
> *insert n into right-sub-tree*

In C, this can be implemented as:

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
  if (self == NULL) { /* found where to put it*/
    self = malloc(sizeof *self);
    self->data_item = n;
    self->left = self->right = NULL;
  } else if (n < self->data_item) {
    self->left = insert_bst_node(self->left, n);
  } else if (n >self->data_item) {
    self->right = insert_bst_node(self->right, n);
  }
  return self;
}
...
```

The following video demonstrates how this works:

▶ Inserting into a BST

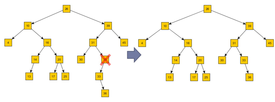
Deleting from a BST

Deleting from a BST is similar to find and insert - we start by recursively searching for the node to delete.

Zero/One Child Case

If the node to delete has zero or one child, deletion is easy. We just replace the pointer to the node to delete with the pointer to the child or NULL if there are no children.

The following example deletes 38 (one child) from the tree.



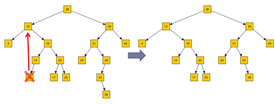
Two Child Case

If the node to delete has two children, we can't just replace it with one of the children, because that would leave the other child unconnected.

Instead, we replace the value stored in the node to be deleted with the value stored in the in-order successor (the node that would be visited next in an in-order traversal). At this point we have two copies of the in-order successor. So, we then recursively call delete on the right subtree to remove th old copy. This second call is guaranteed to be the simpler zero/one child case, since the in-order successor cannot have a left child.

Note: We could use the in-order predecessor, and this would work just as well. But all the examples (and quiz questions) in this unit use the successor.

The following example deletes 10 (two children) from the tree.



To implement the delete function we first need some code to find the in-order successor or, in other words, the minimum node in a sub-tree. This will be helper function only (although you could make it a public function if needed elsewhere), so there is no need for a wrapper function - we can work with the nodes directly. The following code looks very much like linked list code since we are just following the links that go left:

```
BSTNodePtr min_node(BSTNodePtr self) {
  BSTNodePtr current = self;
  while (current->left != NULL) {
    current = current->left;
  }
  return current;
}
```

Now we can write the delete function itself with it's accompanying wrapper:

```
BSTNodePtr delete_bst_node(BSTNodePtr self, int n) {
  if (self == NULL) {
    if (n == self->data_item) { // found item
      if (self->left != NULL && self->right != NULL) {
        // two child case
        BSTNodePtr successor = min_node(self->right);
        self->data_item = successor->data_item;
        self->right = delete_bst_node(self->right, self->data_item);
      } else { // one or zero child case
        BSTNodePtr to_free = self;
        if (self->left) {
          ...
```

The following video talks through this code:

▶ Deleting from a BST

Summary

So, there you have the BST. We said at the start that all operations are $O(height)$ and that for random insertions, the height is $O(\log n)$. If even the average case was also $O(\log n)$ that wouldn't be too bad.

But random insertions are rarely going to represent typical usage, and in fact it will be reasonably common for insertions to be in order, which will result in the worst case.

This would give $O(n)$ for all operations, and it's not just in order insertions that produce this. For example, inserting in this order 6, 26, 10, 25, 16, 20 also gives the worst case.

So, have we done all of this for nothing?

Well, no. A BST is still a very useful data structure and in many cases it will perform close to $O(\log n)$. But furthermore, we can improve on the BST by ensuring that the tree always remains balanced. And that is the topic of the next section.

UNIVERSITY of TASMANIA